

人工智能课程设计——课程作业 1

逻辑推理板块——八皇后问题

一、实验背景

逻辑编程是一种独特的声明式编程典范，它通过设置答案必须匹配的规则和已知事实来解决问题，而非规定具体的计算机执行步骤，其核心过程可概括为“事实 + 规则 = 结果”。在人工智能的发展史中，逻辑编程与 AI 相互促进、相辅相成，早期的人工智能主要以规则和逻辑推理为研究方向，不仅奠定了逻辑编程的基础，而更好、更快的逻辑编程引擎也直接推动了专家系统、知识图谱及自动定理证明等传统 AI 核心领域的突破。

随着数据驱动学习时代的到来，Python 凭借其简洁的语法与强大的生态，已成为人工智能算法的第一语言。在处理离散组合优化及逻辑约束问题 (CLP) 时，将 Python 与逻辑编程（如利用 `kanren` 库）相结合，能够让我们以声明式的方式优雅地表达复杂的约束关系。本实验通过经典的“八皇后问题”，引导我们自主撰写逻辑规则，在实践中深刻体会逻辑编程与传统命令式编程的本质区别，并掌握不同编程范式下的算法性能与复杂度特性。

二、实验内容

本实验的核心研究对象是经典的“八皇后问题”，即在 8×8 的国际象棋棋盘上放置八个皇后，要求任何一个皇后都无法直接吃掉其他的皇后，也就是说，任意两个皇后都不能处于同一条横行、纵行或斜线上。实验要求结合 Python 开发环境和逻辑编程库 `kanren`，分别通过“非逻辑编程穷举+逻辑变量筛选”、“纯逻辑编程动态回溯”以及“自主实现 `neq` 约束算子”等多种创新思路来构建解空间并进行约束求解。最终，还需要将逻辑编程的实现结果与传统的命令式算法（深度优先搜索 DFS 和广度优先搜索 BFS）在运算时间、算法复杂度等方面进行全方位的横向对比，从而深度理解声明式编程与命令式编程的范式差异。

三、实验过程

（1）穷举法+逻辑筛选

八皇后问题要求在 8×8 的棋盘上放置 8 个皇后，使其满足“不同行、不同列、不同对角线”的约束。

该任务规定使用非逻辑编程进行穷举。对于 8 个皇后的位置，可以抽象为 0 到 7 的全排列问题，第 m 个数是 n 表示第 m 行的皇后在第 n 列，这样天保证了皇后之间不同行、不同列的要求。

然后再使用逻辑变成进行筛选。在 `kanren` (LogPy) 中，通过将这 40320 种排列录入到逻辑关系事实库 (Facts) 中。然后，构建一个底层的逻辑目标 (Goal) 函数，让 `kanren` 的推理引擎在遍历这些事实时，动态调用传统的 Python 对角线冲突检查函数，从而筛选出合法解。

代码实现：

```
import time
from itertools import permutations
```

```

from kanren import Relation, facts, run, var

# =====
# 1. 非逻辑编程方式：穷举所有可能的排列 (Col Positions)
# =====
# 使用 permutations 天然保证了任意两个皇后不在同一列，也不在同一行
all_permutations = list(permutations(range(8)))
print(f"【非逻辑编程】穷举生成的排列总数：{len(all_permutations)} 种")

# =====
# 2. 逻辑编程方式：筛选出符合对角线规则的解
# =====

# 定义一个检查对角线冲突的辅助函数
def is_valid_diagonal(perm):
    """
    检查一个排列是否满足对角线不冲突。
    若两个皇后坐标为 (i, perm[i]) 和 (j, perm[j]),
    则满足  $\text{abs}(i - j) \neq \text{abs}(\text{perm}[i] - \text{perm}[j])$ 
    """
    for i in range(8):
        for j in range(i + 1, 8):
            if abs(i - j) == abs(perm[i] - perm[j]):
                return False # 发生对角线冲突
    return True

# =====
# 3. 利用 kanren 进行逻辑筛选
# =====

# 声明一个逻辑关系：代表“这是一个生成的排列”
permutation_rel = Relation()

# 把【所有】未经过滤的穷举排列（40320 种）作为事实录入数据库
facts(permutation_rel, *[(p,) for p in all_permutations])

# 声明逻辑变量
q_solution = var()

# 通过底层构建一个合法的 kanren 目标 (Goal) 来替代 condition
def diagonal_goal_func(var_sol):
    def goal(sub):
        # 获取当前逻辑变量被绑定（实例化）后的具体值
        val = sub.get(var_sol, var_sol)

```

```

        # 如果变量已经绑定了具体的排列，并且通过了对角线检查，则保留该状态
        if isinstance(val, tuple) and is_valid_diagonal(val):
            yield sub
    return goal

print("\n 正在启动 kanren 逻辑推理与筛选...")
start_logic = time.time()

# 运行推理：寻找同时满足“是排列事实”且“通过自定义对角线逻辑目标”的解
answers = run(0, q_solution,
              permutation_rel(q_solution),
              diagonal_goal_func(q_solution))

end_logic = time.time()

# =====
# 4. 结果输出
# =====
print(f"【逻辑编程】筛选出的合法八皇后解总数：{len(answers)} 种")
print(f"【逻辑编程】通过自定义逻辑目标筛选耗时：{end_logic -
start_logic:.6f} 秒")
    运行结果为：
    【非逻辑编程】穷举生成的排列总数: 40320 种
    正在启动 kanren 逻辑推理与筛选...
    【逻辑编程】筛选出的合法八皇后解总数: 92 种
    【逻辑编程】通过自定义逻辑目标筛选耗时: 0.458580 秒

```

（2）纯逻辑编程

解决思路：纯逻辑编程的精髓在于彻底摆脱对外部穷举数据的依赖。我们直接向 kanren 引擎声明 8 个逻辑变量 $q_0, q_1, \dots, q_7$ 。利用 `membero(q_i, range(8))` 分别声明它们的值域。为了避免盲目组合带来的组合爆炸，我们采用**交错约束网**的策略：每为一行皇后 q_i 声明值域后，立刻让它与前面所有已经声明的皇后 ($q_0 \dots q_{i-1}$) 建立冲突检查目标。这样，kanren 引擎在隐式深度搜索时，只要发现当前行的列号与前几行冲突，就会立刻触发回溯，从底层剪掉不合法的子树，实现边赋值边剪枝。

实现代码：

```

def solve_queens_pure_logic():
    # 1. 声明 8 个逻辑变量，分别代表 0~7 行皇后所在的列
    queens = [var(f'q_{i}') for i in range(8)]

    # 2. 约束：每个皇后的列坐标必须在 0~7 之间
    # membero(x, coll) 表示 x 必须是迭代器 coll 中的一个元素
    domain_goals = [membero(q, range(8)) for q in queens]

```

```

# 3. 约束：任意两个皇后不能在同一列，也不能在同一对角线
constraint_goals = []

# 构建一个自定义的逻辑目标，用于动态检查两个已经绑定值的皇后是否冲突
def safe_pair_goal(q_i, q_j, i, j):
    def goal(sub):
        # 获取当前状态下，两个逻辑变量的实例化值
        val_i = sub.get(q_i, q_i)
        val_j = sub.get(q_j, q_j)

        # 如果两个变量都已经被赋予了具体的数字，则进行冲突检查
        if isinstance(val_i, int) and isinstance(val_j, int):
            # 检查列冲突
            if val_i == val_j:
                return

            # 检查对角线冲突
            if abs(val_i - val_j) == abs(i - j):
                return

            # 如果没冲突，或者变量还没完全绑定，则保留当前状态继续向下搜索
            yield sub
    return goal

# 双重循环，为任意两行（i, j）之间建立逻辑约束目标
for i in range(8):
    for j in range(i + 1, 8):
        constraint_goals.append(safe_pair_goal(queens[i], queens[j],
i, j))

# 4. 将所有的值域约束和冲突约束合并为一个庞大的“逻辑约束网”
# 注意：在 kanren 中，约束的顺序至关重要。
# 我们交叉放置 membero 和 safe_pair_goal，让 kanren 赋值一个就立刻检查一个（即边生成边剪枝）
all_goals = []
for i in range(8):
    all_goals.append(domain_goals[i])
    # 每多确定一个皇后的位置，就和之前所有已确定的皇后进行冲突对比
    for j in range(i):
        all_goals.append(safe_pair_goal(queens[j], queens[i], j, i))

print("正在启动纯逻辑编程求解（动态约束传递与回溯）...")
start_time = time.time()

# 5. 运行 kanren 求解引擎

```

```
# *all_goals 将整个约束网络解包传入，queens 元组作为我们要获取的答案形式
solutions = run(0, queens, *all_goals)
```

```
end_time = time.time()
```

```
print(f"【纯逻辑编程】求解出的合法解总数: {len(solutions)} 种")
```

```
print(f"【纯逻辑编程】耗时: {end_time - start_time:.6f} 秒\n")
```

运行结果为:

【纯逻辑编程】求解出的合法解总数: 92 种

【纯逻辑编程】耗时: 4.218238 秒

(3) neq 函数实现及应用与对比

Neq 函数设计

在逻辑编程中，原生的 eq 负责建立绑定关系。而 neq(x, y) 作为不等于约束，不能简单地用 Python 的 != 返回布尔值，因为在引擎推理初级阶段，逻辑变量 x 和 y 尚未绑定具体数值。因此，neq 必须是一个逻辑目标生成器，通过闭包截获环境状态 sub:

如果 x 和 y 中存在未绑定的逻辑变量，允许其通过（保持约束传递）；

只有当 x 和 y 均被实例化为具体整数时，才激活校验：相等则中断当前流（返回空），不相等则保留当前流（yield sub）。

实现代码：

```
import time
```

```
from kanren import run, var, membero
```

```
# =====
```

```
# 1. 自主实现 neq 函数
```

```
# =====
```

```
def neq(x, y):
```

```
    """自定义逻辑不等于目标"""
```

```
    def neq_goal(sub):
```

```
        val_x = sub.get(x, x)
```

```
        val_y = sub.get(y, y)
```

```
        if not isinstance(val_x, type(x)) and not isinstance(val_y,
type(y)):
```

```
            if val_x == val_y:
```

```
                return
```

```
            yield sub
```

```
    return neq_goal
```

```
# =====
```

```
# 2. 自主实现对角线不相等约束
```

```
# =====
```

```
def neq_diagonal(q_i, q_j, i, j):
```

```
    """自定义对角线不相等目标: |q_i - q_j| != |i - j|"""
```

```

def diagonal_goal(sub):
    val_i = sub.get(q_i, q_i)
    val_j = sub.get(q_j, q_j)
    if not isinstance(val_i, type(q_i)) and not isinstance(val_j,
type(q_j)):
        if abs(val_i - val_j) == abs(i - j):
            return
    yield sub
return diagonal_goal

# =====
# 3. 结合自定义函数求解八皇后
# =====
def solve_queens_with_custom_neq():
    queens = [var(f'q_{i}') for i in range(8)]

    # 动态构建交错的约束流（赋值一个，立刻检查与之相关的列和对角线）
    all_goals = []
    for i in range(8):
        all_goals.append(membero(queens[i], range(8)))
        for j in range(i):
            # 使用自主实现的 neq 函数约束列号
            all_goals.append(neq(queens[j], queens[i]))
            # 使用自主实现的对角线函数约束对角线
            all_goals.append(neq_diagonal(queens[j], queens[i], j, i))

    print("正在启动包含自定义 neq 函数的逻辑编程求解...")
    start_time = time.time()

    solutions = run(0, queens, *all_goals)

    end_time = time.time()

    print(f"【自定义 neq 逻辑编程】求解出的合法解总数: {len(solutions)} 种")
    print(f"【自定义 neq 逻辑编程】耗时: {end_time - start_time:.6f} 秒\n")

if __name__ == "__main__":
    solve_queens_with_custom_neq()
    运行结果:
    【自定义 neq 逻辑编程】求解出的合法解总数: 92 种
    【自定义 neq 逻辑编程】耗时: 8.468721 秒

```

(4) BFS / DFS 算法实现

BFS 和 DFS 是命令式编程的方法，也是更为常见且效率更高的方法。

实现代码如下：

```
# =====
# 1. 经典 DFS（深度优先搜索 / 回溯法）实现
# =====
def solve_dfs():
    solutions = []

    def is_safe(board, row, col):
        # 检查前面的每一行，看是否与当前位置 (row, col) 冲突
        for i in range(row):
            # board[i] == col 检查列冲突
            # abs(board[i] - col) == abs(i - row) 检查对角线冲突
            if board[i] == col or abs(board[i] - col) == abs(i - row):
                return False
        return True

    def dfs(row, current_board):
        if row == 8:
            solutions.append(tuple(current_board))
            return

        for col in range(8):
            if is_safe(current_board, row, col):
                current_board.append(col) # 放置皇后
                dfs(row + 1, current_board) # 递归下一行
                current_board.pop()        # 回溯（撤销放置）

    start_time = time.time()
    dfs(0, [])
    end_time = time.time()

    print(f"【经典 DFS】求解出的合法解总数：{len(solutions)} 种")
    print(f"【经典 DFS】耗时：{end_time - start_time:.6f} 秒\n")
    return solutions

# =====
# 2. 经典 BFS（广度优先搜索）实现
# =====
def solve_bfs():
    solutions = []
    # 队列里初始存放一个空棋盘状态 []
    queue = deque([[]])

    def is_safe(board, row, col):
```

```

    for i in range(row):
        if board[i] == col or abs(board[i] - col) == abs(i - row):
            return False
    return True

start_time = time.time()

while queue:
    current_board = queue.popleft()
    current_row = len(current_board)

    # 如果已经成功放置了 8 个皇后，说明找到了一个合法解
    if current_row == 8:
        solutions.append(tuple(current_board))
        continue

    # 尝试在当前行（current_row）的每一列放置皇后
    for col in range(8):
        if is_safe(current_board, current_row, col):
            # 产生一个新状态并推入队列
            queue.append(current_board + [col])

end_time = time.time()

print(f"【经典 BFS 】求解出的合法解总数: {len(solutions)} 种")
print(f"【经典 BFS 】耗时: {end_time - start_time:.6f} 秒\n")
return solutions

```

运行结果:

【经典 DFS 】求解出的合法解总数: 92 种

【经典 DFS 】耗时: 0.004000 秒

【经典 BFS 】求解出的合法解总数: 92 种

【经典 BFS 】耗时: 0.004997 秒

（5）算法对比

求解方法	运行时间	时间复杂度	空间复杂度	编程范式类型
穷举+逻辑筛选	0.45 秒	$O(N!)$	$O(N!)$	混合范式
纯逻辑编程	4.17 秒	远优于最坏 $O(N!)$	$O(N)$	逻辑编程
自定义 neq	8.46 秒	远优于最坏 $O(N!)$	$O(N)$	逻辑编程
DFS	0.004 秒	远优于最坏 $O(N!)$	$O(N)$	命令式编程
BFS	0.005 秒	远优于最坏 $O(N!)$	$O(W)$	命令式编程

1. 运算时间与执行效率分析

从实验测试数据可以明显看出，不同求解方法在运行时间上呈现出高达三个数量级（上千倍）的巨大反差：

传统命令式算法(DFS/BFS)具有压倒性的速度优势。深度优先搜索(DFS)与广度优先搜索(BFS)的运行时间均稳定在 5~6 毫秒左右。这是因为它们完全基于 Python 原生列表对象操作，控制流直接由底层的 C 语言循环优化支撑，且在搜索过程中通过条件判断进行了极其高效的实时剪枝，没有任何多余的中间件或框架开销。

基于 kanren 的逻辑编程方法运行效率相对较低。纯逻辑编程求解耗时约为 4.17 秒，而引入自定义 neq 函数后，由于增加了频繁的类型检查(isinstance)与闭包函数调用，常数项开销进一步放大，耗时延长至 8.46 秒。这主要是因为 kanren 作为一个微型 Prolog 引擎，其底层需要不断维护复杂的符号替换表(Substitution Map)，并在状态回溯时频繁复制环境流(Stream)，这种高度的抽象包装带来了高昂的计算代价。

“穷举 + 逻辑筛选”表现居中。耗时约为 0.45 秒。其全排列生成阶段依赖于 Python 底层经 C 优化的内置迭代器，速度极快，但由于后续必须让逻辑引擎强行遍历多达 40,320 种事实数据，拉低了整体的性能表现。

2. 算法复杂度与资源消耗分析

时间复杂度：尽管所有算法在最坏情况下（无剪枝盲目搜索）的时间复杂度均为 $O(N!)$ ，但在实际执行中，**纯逻辑编程、自定义 neq 法、DFS 以及 BFS 均实现了不同程度的动态剪枝**。通过“边赋值、边前向检查”的策略，它们在搜索树的较高层级就切断了不合法的分支，使得实际搜索空间远优于最坏情况。相比之下，**穷举筛选法**由于必须完整生成并扫描全部排列，时间复杂度硬性锁死在 $O(N!)$ ，缺乏动态扩展能力。

空间复杂度：在空间开销方面，**DFS、纯逻辑编程与自定义 neq 法最为优秀**，其空间复杂度为 $O(N)$ ，内存占用仅与棋盘的行数（即搜索树的深度）成线性正比。**BFS 的空间复杂度取决于搜索树的最大宽度 $O(W)$** ，由于需要利用队列维护每一层所有可能存在的合法状态，内存压力会随着问题规模的扩大呈指数级爆炸。而**穷举筛选法**需要将海量的组合全部序列化存入内存中的事实库，空间复杂度高达 $O(N!)$ ，最容易导致内存溢出。

3. 编程范式维度的深层思考

本次对比不仅是性能的较量，更是命令式(Imperative)与声明式(Declarative)两种编程范式的碰撞：

经典 DFS/BFS (命令式)：程序员必须像指挥官一样，精确地告诉计算机“如何去写循环”、“何时入队出队”以及“如何手动执行回溯”。这种范式虽然编写复杂、容易出错，但带来了极致的执行效率和底层控制力。

逻辑编程 (声明式)：程序员只需要像法律制定者一样，清晰地声明变量的值域(membero)以及变量间不可逾越的数学约束规则(neq)。至于计算机“如何去搜索”、“如何去回溯”，全权交由引擎隐式完成。这种范式极大地解放了开发者的逻辑思维，代码极其精简、易于维护，但其代价则是将优化压力转嫁给了底层的逻辑推理引擎。

四、改进方向

虽然本实验通过多种编程范式成功解决了经典的八皇后问题，并自主实现了逻辑不等于约束，但从测试数据中可以看出，纯逻辑编程在执行效率上仍有较大

的优化空间。针对其底层机制与算法结构，本实验提出以下三个具有实际工程价值的改进与优化方向：

1. 引入动态启发式搜索策略（如 MRV 机制）

目前在 kanren 逻辑引擎中构建的约束网络，是按照棋盘行号（从 q_0 到 q_7 ）的固有顺序进行静态赋值搜索的。这种盲目的按序搜索没有考虑各行当前剩余可选位置的紧迫程度。

优化方向：可以借鉴约束满足问题（CSP）中经典的**最少剩余值（Minimum Remaining Values, MRV）启发式策略**。在逻辑引擎每次尝试为下一个皇后赋值前，动态计算当前所有未赋值行中“哪一行的合法剩余列坐标最少”，并优先为该行进行推导绑定。这种“最紧迫优先”的策略能够让搜索树在极高层级就触发矛盾并进行剪枝，从而呈指数级减少无效的回溯次数。

2. 构建主动值域擦除的约束传播引擎（Constraint Propagation）

本实验在任务 (3) 中自主实现的 `neq` 和 `neq_diagonal` 约束算子，本质上属于“被动式的状态过滤器”。只有当两个皇后变量都被引擎具体实例化为具体的整数后，才会激活冲突判定并拦截错误流。此时，引擎已经做了大量的无用功。

优化方向：更高级的约束逻辑编程（CLP）系统（例如 Prolog 语言中的 `clpfd` 核心库）应当具备前向推导（Forward Checking）能力。即当第一行的皇后 q_0 确定绑定在第 0 列时，引擎应当产生约束传播，主动将 0 这一列以及相关的对角线斜率坐标，从剩余所有未赋值皇后（ $q_1 \dots q_7$ ）的可用值域（Domain）中直接擦除。如果在 kanren 库的流控制（Stream）底层实现这种动态域缩减机制，纯逻辑编程的运行效率将产生质的飞跃，有望在常数项开销上追平甚至超越传统的命令式回溯算法。

3. 利用多进程（Multiprocessing）并发优化解空间检索

在处理更大规模的 N 皇后问题时（例如 $N \geq 12$ ），解空间和搜索树节点的数量将呈几何级数爆炸。目前不论是经典回溯法（DFS）还是逻辑编程的流推导，所有的状态匹配和剪枝过程均在 CPU 的单核单线程上串行执行，未能充分释放现代多核硬件的算力。

优化方向：可以采用“分治与并发”的思路。在算法启动初期，将第一行皇后分别放置在不同列的几种根状态（如 $q_0 = 0, q_0 = 1$ ）作为独立子任务，利用 Python 的 `multiprocessing` 库分发给操作系统的多个 CPU 核心。每个核心独立运行一套逻辑推理引擎或回溯进程，最后对计算出来的合法流进行合并。这种多进程并发优化在面对高阶约束问题时，能使总运算时间随核心数线性下降。

五、调试过程与体会

1、环境依赖调试

由于 kanren (LogPy) 属于 Python 的第三方轻量级声明式逻辑编程库，在代码编写初期，首要任务是确保开发环境、解释器路径以及库的依赖正常。为此，专门编写了独立测试脚本 `test.py` 进行环境断言与基准功能验证。

调试目的：验证 Python 虚拟环境路径、检查 kanren 库是否成功安装，并测试逻辑变量 `var()` 与推理引擎 `run()` 的最简逻辑绑定（如寻找 x 使得 $x == 5$ ）是否符合预期。

调试核心代码：

```

8  try:
9      from kanren import run, eq, var, membero
10     print("✓ kanren 导入成功")
11
12     # 简单测试
13     x = var()
14     result = run(0, x, eq(x, 5))
15     print(f"\n逻辑推理测试: 找出 x 使得 x == 5")
16     print(f"结果: {result}")
17
18     if result == (5,):
19         print("✓ 逻辑推理正确")
20     else:
21         print("X 逻辑推理异常")
22
23 except ImportError as e:
24     print(f"X kanren 导入失败: {e}")
25     print("\n请先安装 kanren:")
26     print("  pip install kanren")

```

2、自定义算子调试

在初步设计 `neq(x, y)` 函数时，直接使用了 Python 的 `val_x != val_y` 进行布尔判断，导致 `kanren` 推理引擎在初期（变量尚未实例化、仍为 `var` 符号时）直接误判并中断流。

解决方法：通过打印环境状态 `sub`，意识到必须利用闭包截获环境，并通过 `isinstance(val_x, type(x))` 来判断变量是否真正被实例化，只有在双方都实例化时才激活校验，否则允许其通过（保持约束传递）。